{{CAMPAIGN_NAME}} Internship Campaign

Build guide and user manual for Claude Code

How the whole system works, how to build it from nothing on a Windows PC, how to use it every day, and how to fix it when something breaks.

{{FULL_NAME}} · {{SCHOOL}}, {{UNIVERSITY}} · rules version 30, 31 October 2026 · if this guide and `CLAUDE.md` ever disagree, `CLAUDE.md` wins.

1 What this system is, in one page
2 How it works: the five ideas that explain everything
3 Build it from scratch: the kit and the complete path
4 The setup prompt: what it does and what it asks
5 Check that it works
6 Using it every day
7 Every command
8 The safety rules: sending, forms, holds
9 What can go wrong, and how to fix it
10 Reference: files, paths, control commands

CHANGED ON 4 AND 6 OCT 2026

Nothing is scheduled any more (4 Oct): you start every command yourself. Runs have no row, search, helper or spending limit (6 Oct): the Checker asks how many rows (any number or ALL), the Applier asks which method (email, portal or both), for Portal which portal(s) from a list with eligible rows per portal (17 Oct), and how many applications of each (14 Oct), the Hunter covers every available opportunity in a country. Every command updates the tracker after every row. Only email stays capped.

1. What this system is, in one page

An internship search for one final-year engineering student, run by Claude Code on the PC. Each command is started by hand and, once started, Claude:

- **finds** internship opportunities in one country at a time (companies and startups first, then labs, universities and job boards),
- **checks** each one: is it still open, who is the right contact, will they take a non-EU student and sign {{SCHOOL}}'s internship agreement,
- **applies**: writes a tailored CV and cover letter for each one, submits company portal forms, and sends application emails,
- and **reads the mailbox**: files every reply, updates each application's status, answers the simple emails, sends due follow-ups, and prepares options for the important ones.

The student only does what a machine should not: answers interview invitations and offers, clicks LinkedIn "Easy Apply", logs in where a password is needed, and answers the rare form question the system cannot answer truthfully.

What runs it

Claude Code (the command-line Claude) on the Windows PC. You type a command; nothing runs on a schedule.

What it remembers with

One spreadsheet file, `tracker.csv`. Every opportunity is one row. No chat history is needed.

What it works through

Your Chrome (already logged in to your sites), your Gmail, Google Drive and Calendar, the web, and Python scripts.

What keeps it safe

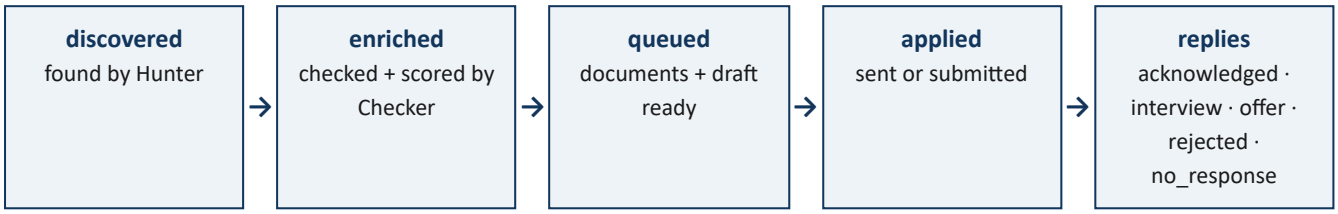
Hard rules in `CLAUDE.md`, a 50-email daily cap, a "holds" list, a saved answers file for forms, and checks before every email.

2. How it works: the five ideas that explain everything

Idea 1. The memory is a file, not a conversation

Every run is a **fresh** Claude session that knows nothing. It reads `02_tracker\tracker.csv`, works on some rows, writes the results back, writes a report, and ends. Closing a session loses nothing. Every command writes the tracker right after each row it works on, so a run cut short loses nothing.

Each opportunity row moves through a fixed set of statuses:



Side exits: **screened_out** / **expired** (dead: moved to `seen.csv` so it is never found again) and **blocked** (needs a human: moved to `blocked.csv`, your manual queue).

Idea 2. One job per run

Each kind of work is a separate "run" with its own instructions, a slash command. A Hunter never applies, an Applier never hunts. This keeps every run small and predictable.

Run	Its one job	Reads	Writes
Hunter	Find every opportunity in one country, then the next (country_priority.csv); companies and startups first, labs and universities still covered	the web, job boards, {{SCHOOL}} registers	new rows: discovered / screened_out
Checker	Verify contacts, visa, agreement; score; set priority A/B/C	organisation sites	discovered → enriched
Applier	Write CV + letter, submit portals, send emails	enriched and blocked rows	→ applied, or → blocked
Inbox	File and answer mail, send due follow-ups, expire dead rows	Gmail, tracker	statuses, responses.csv, waiting_list.csv, drafts, follow-ups, Calendar
Cleanup	Monthly archive of dead rows	tracker	seen.csv

Idea 3. You start each run; no run has a limit

Nothing is scheduled. You open Claude Code in the folder and type the command you want. `gate.py` can tell you which commands have work. No run has a row, search, helper or spending cap; only email is capped.

Command	How much it covers	Asks first
/pfe-inbox	All PFE mail; follow-ups max 15 and simple replies max 10 per run	no
/pfe-send-held	Drafts held back by yesterday's 50-email cap	no
/pfe-checker [n or ALL]	As many rows as you choose, no maximum	how many rows (10, 25, 50, ALL or any number)

/pfe-applier [n email] [n portal]	As many applications as you choose, by email, portal or both, no maximum; started without a method or number it asks first, before anything else, even when an unfinished run left a PROGRESS file (that file then only skips the IDs already done)	which method(s); for Portal, which portal(s); then how many applications of each (10, 25, 50, ALL or any number)
/pfe-cleanup	Monthly archive of dead rows	no
/pfe-hunter [country]	Every available opportunity in the country, every source and track, until it is exhausted	no

The tracker is updated after every row. Every command writes each row (and each action on it: status, enrichment, documents, submission, send, reply, block) through tracker_tools the moment it is done, never at the end. Big runs can reach your Claude plan's 5-hour usage limit or fill the session: the run then stops, everything done so far is saved, and running the same command again continues (the Applier from its PROGRESS file). Every report says how many rows were covered and how many remain.

How the Hunter chooses countries

You never choose. `01_sources\country_priority.csv` ranks the countries you gave the setup (at least 10) by fit with your profile (language, visa for a {{NATIONALITY}} student, match with your tracks, realistic chance). The Hunter stays on one country, run after run, going through all its offers and organisations, every source and every track (track ALL), with no row or search limit, until the country is exhausted (every source and track done and its last 20 searches found no new row). Then it marks it exhausted and moves to the next. After the last one, it researches the next 15 most compatible countries itself, adds them to the file, and carries on. Each row of the file shows the status (pending, active, exhausted), runs, rows added and last run.

Idea 4. Instructions live in text files

- `CLAUDE.md` holds the **rules** that apply to every run: who the student is, what the internship must be, what may never be invented, how to write, how to use the browser. Claude Code loads it automatically in this folder.
- `.claude\commands\pfe-*.md` hold each run's **job**: one Claude Code prompt per command (version 3 merged the original Cowork prompts and their Claude Code adaptations into one text). `$ARGUMENTS` is whatever you type after the command.
- `00_profile\` holds the **facts**: verified profile, defensible claims, tracks, keywords, and the saved form answers.

Idea 5. Machines do volume, people do decisions

Portals and emails go out automatically, but only after checks (section 8). Anything with consequences, such as an interview, an offer, salary, a visa question or a password, stops and waits for you, either in `/pfe-send` or in `blocked.csv`.

3. Build it from scratch: the complete path

Everything comes in one folder, **sniper**, the transfer package. Copy it to the new PC (for example to `Desktop\pfe workflow\sniper`) and open `00_START_HERE.md`. It holds:

Folder or file	Role
`01_install\`	Prerequisites with tested versions, `requirements.txt`, `package.json` and `install.ps1` (installs the Python libraries and checks the tools; it schedules nothing).
`02_setup_prompt\`	The one prompt a new Claude Code session runs to build or restore everything, `answers_TEMPLATE.json` and `fill_placeholders.py`.
`03_kit\`	The ready-to-install files: `campaign\`, `interviews\`, `my documents\`, with placeholders such as `{{FULL_NAME}}` instead of personal details (`PERSONALISE.md` lists them).
`04_runbook_and_guides\`	The runbook (architecture, every prompt, and the full text of every script and config file between "BEGIN FILE" and "END FILE"), this manual, and Friendly Setup and Usage.
`05_build_sources\`	The sources that rebuild the kit, the runbook and the guides. The author's `/forever` keeps everything in step.

The setup builds one ROOT folder: `campaign\` (the workspace, where Claude Code is opened), `interviews\`, `my documents\`, `guides\`, next to `sniper\`. It decides the path by itself:

Path A: restore (the campaign exists)

`ROOT\campaign` with tracker.csv, documents and reports is already there (new PC with a synced folder, reinstall, broken config). The setup installs the kit and keeps every piece of campaign data and every decision file.

Path B: brand-new campaign

No tracker yet. The setup interviews you once, creates the folders, fills every placeholder, asks for your source documents, then runs the three one-time prompts from the runbook: S0 PROFILE, CAMPAIGN INIT, SEED IMPORT (only if you have registers of past placements).

Follow the steps in order. Each ends with a ✓ **check**: do not continue until it passes.

Part 1. Install the tools (both paths, about 45 minutes)

1. **The folder.** Copy `sniper` where your campaign will live. If it is in OneDrive, sign in and let it sync (OneDrive can lock a file for a few seconds; the tools retry).
   ✓ `00_START_HERE.md` opens.

2. **Claude Code.** Install it (claude.com/claude-code), open PowerShell, type `claude`, and log in with the claude.ai account that holds your connectors (`/login`). It installs to `C:\Users\<you>\.local\bin\claude.exe`.
   ✓ `claude --version` prints a version.

3. **Python 3.12 and Node.js.** Python from python.org, "install for me only" (goes to `C:\Users\<you>\AppData\Local\Programs\Python\Python312\`); Node.js LTS from nodejs.org (for the interview decks). Then, in PowerShell in `sniper\01_install`:

   ```
   powershell -ExecutionPolicy Bypass -File .\install.ps1
   ```

   It installs the libraries of `requirements.txt` with the full Python path and checks Word, Chrome and Node. Always call Python by that full path: typing `python` alone opens the Microsoft Store.
   ✓ It ends with "all checks passed".

4. **Microsoft Word.** The document generator turns each CV and letter into PDF through Word. Open Word once to finish its first-run screens.
   ✓ Word opens with no dialog.

5. **Chrome and Claude in Chrome.** In the Chrome profile you will use for the campaign, install the **Claude in Chrome** extension and set: site access **On all sites**, permission mode **Act without asking**. Log in to Gmail, LinkedIn, Indeed, JobTeaser, Welcome to the Jungle, EURAXESS, and every portal where you have a candidate account. Tick "stay signed in".
   ✓ Each site shows you logged in.

6. **Connectors.** claude.ai → Settings → Connectors: **Gmail**, **Google Drive**, **Google Calendar** (the Google account you apply from) and **Indeed**. In Gmail create the label you want campaign mail filed under.
   ✓ The four show "Connected".

7. **Path B only: your source documents.** Have ready: master CV (.docx and .pdf), research and internship reports, recommendation letters, your school's internship agreement templates, and any register of past placements. The setup tells you where to put them (`ROOT\my documents\`). These are the only files Claude cannot create.

Part 2. Run the setup prompt (both paths, about an hour)

1. Open PowerShell in the folder that holds `sniper` and start Claude Code with Chrome enabled:

   ```
   claude --chrome
   ```

   Accept "trust this folder". Type `/chrome` → **Enabled by default**, and choose the browser of your campaign profile.

2. Open `sniper\02_setup_prompt\PFE_CLAUDE_CODE_SETUP_PROMPT.md`, copy everything **below the line** and paste it. It asks you once, in one message, for everything it cannot read itself (name, school, degree, dates, languages, contact, Chrome profile, Gmail label, countries, tracks, instruments and skills, form answers), or reads `answers.json` if you filled it from `answers_TEMPLATE.json`.

3. Let it work. It reads the state, picks Path A or B, creates the folders, fills every placeholder (`fill_placeholders.py` must say "placeholders left: 0"), runs the one-time prompts on Path B, runs a smoke test and tells you which command to run first. It schedules nothing. Path B stops once to ask for your source documents.

4. **Finish what Claude Code refuses to do itself.** Its safety check may refuse to change Claude's own permissions. The setup then gives you the exact command:

   1. **Permissions:** `! notepad ".claude/settings.json"`, paste the version it prints (also in section 10), save, and ask "check settings.json is valid".
   2. **First run:** open a new session in `ROOT\campaign` and type the command it names (`/pfe-inbox`, or `/pfe-hunter` on a brand-new campaign).

TIP: THE ! PREFIX

Inside Claude Code, `! command` runs a command in Git Bash, which eats backslashes. Always write paths with **forward slashes**: `scheduler/gate.py`.

What no file can carry: check these on every new machine

Item	How to set it	How the system tells you when it is wrong
Claude Code login	`claude`, then `/login` with the account that holds the connectors	The first command fails to start
Connectors (Gmail, Drive, Calendar, Indeed)	claude.ai → Settings → Connectors, the Google account you apply from	The run report names the failing connector. Indeed falls back to indeed.* in Chrome.
Chrome profile and extension	Extension in the campaign profile, "On all sites", "Act without asking"; `/chrome` → Enabled by default	`scheduler/preflight.py` (machine check, run by hand)
Gmail session in Chrome	Sign in to Gmail in that profile whenever Google asks	Email rows BLOCKED "SESSION EXPIRED: GMAIL" in the Applier report
Job-site and portal logins	Sign in once in that profile, tick "stay signed in"	Rows BLOCKED "SESSION EXPIRED" or "LOGIN_WALL"
Python libraries, Node, Word	Part 1 steps 3 and 4	`scheduler/preflight.py`
Gmail label {{GMAIL_LABEL}}	Create it once in Gmail	The Inbox report

NEVER ON AN EXISTING CAMPAIGN

The one-time prompts S0 PROFILE, CAMPAIGN INIT and SEED IMPORT build a new tracker. The setup runs them only on Path B. Never paste them yourself into a session of an existing campaign.

4. The setup prompt: what it does and what it asks

File: `sniper\02_setup_prompt\PFE_CLAUDE_CODE_SETUP_PROMPT.md` (rules version 24). The same text is in the shared runbook in sniper, section 4 (not in your own locked copy).

Step	What it does	You see
Source of the files	Copies from `sniper\03_kit\`. If a kit file is missing, extracts it from the shared runbook in sniper (between "BEGIN FILE" and "END FILE"), only before LOCKDOWN. Your own runbook in guides is a locked copy without file contents.	
Rule zero Data is sacred	Never resets tracker.csv, responses, seen, blocked, backups, profile files, documents, reports or interview folders. Keeps your holds.csv, country_priority.csv, FORM_ANSWERS.md, personalisation_map.json and profile_deck.json if they exist. Backs up every config file before replacing it. Sends and submits nothing.	
0 Read the state	Checks ROOT, the tracker, the tools (Claude Code, Python libraries, Node, Word, Chrome); lists the Chrome profiles with the extension; one read test per connector; lists any scheduled task for you to remove.	A STATE line, Path A or B
1 Interview	Asks once, all together, for every value it cannot read itself; writes `answers.json`.	One message of questions
2 Create the folders (B)	Copies the kit to ROOT, fills the placeholders, writes country_priority.csv, installs the deck builder, asks for your source documents.	"placeholders left: 0"
3 Install or update the kit	Path A: copies the kit over the workspace following rule zero and fills it from your personalisation_map.json.	Files installed / kept
4 One-time prompts (B)	S0 PROFILE, CAMPAIGN INIT, SEED IMPORT (if you have registers); then the profile-based answers and the deck's shared slides.	One report per prompt
5 Smoke test	tracker summary and Excel export, gate.py and preflight.py, build.py on a temporary copy (one-page CV and letter), one test deck, no placeholder left, 9 commands present.	One line per check
6 Lockdown	Keeps your answers.json and any file of yours left in the package (moved to my documents\from sniper), marks CLAUDE.md, the commands, settings.json and your guides read-only, then deletes sniper and its .zip for good (not through the Recycle Bin). Nothing personal is deleted.	What was kept, locked and deleted
7 First run	Schedules nothing. Tells you which command to type in a new session.	What to run first

The one-time prompts (Path B)

Prompt	Creates
S0 PROFILE	From your documents: PROFILE.md (team work and your own work kept apart), DEFENSIBLE_CLAIMS.md, TRACKS.md (ranked by realistic chance), KEYWORDS.md (per track, several languages), SOURCES.md (60+ places to search).

CAMPAIGN INIT	The empty tracker files with their headers, a tested tracker_tools, CV_MASTER_V2.md and .docx (following the build.py paragraph contract), LETTER_MASTER.md, source_registry.csv, the contact line, instruments and course list in personalisation_map.json.
SEED IMPORT	Optional (EXAMPLE: only if your school gives registers of past placements): tracker rows from them (hosts that already signed an agreement with {{SCHOOL}}) and insat_host_patterns.md.

5. Check that it works

	Check	How
☐	Kit installed	The setup's smoke test says "placeholders left: 0" and lists the 9 command files.
☐	Commands appear	Type `/` in Claude Code: 9 commands listed (eight `/pfe-` and `/ask-me`). The sniper folder is gone: the setup deleted it on purpose.
☐	Gate works	`! "%LOCALAPPDATA%/Programs/Python/Python312/python.exe" scheduler/gate.py` prints one JSON line.
☐	Inbox test passed	`05_reports\inbox_<date>.md` exists; no "permission denied"; new files in `02_tracker\backups\`; row count did not drop.
☐	Nothing scheduled	`! powershell -c "Get-ScheduledTask -TaskPath '\PFE2027\'"` shows nothing.
☐	Chrome is the right profile	`/chrome` shows the browser whose profile is logged in to your sites.
☐	Gmail account in Chrome	In Chrome, which `mail.google.com/mail/u/N/` shows the {{SCHOOL}} account? If not u/0, tell Claude "Gmail compose is u/N".

6. Using it every day

Every day

Open Claude Code in the folder and run `/pfe-inbox`, then `/pfe-send-held`, `/pfe-checker` (answer how many rows, or type ALL) and `/pfe-applier` (tick email, portal or both, then how many of each), and `/pfe-hunter` when few rows wait at discovered. A run that stops on the usage limit continues when you run it again.

When a report says "Decisions waiting"

Open Claude Code in the folder, type `/pfe-send`, answer e.g. `D: PFE-0042 option B`. 5 minutes.

Once a week

Your manual queue, `02_tracker\blocked.csv`: Easy Apply links, NEEDS_ANSWER questions, login walls. Apply by hand, then tell Claude "PFE-xxxx submitted". 30 minutes.

Interview booked

`/pfe-interview-prep <their email>`. Then settle its "CHECK BEFORE THE INTERVIEW" list yourself.

Any time

Open `02_tracker\tracker.xlsx` (not the .csv). Its first sheet, **dashboard**, has four pie charts refreshed after every run: all rows by status, and what was sent this week, last week and two weeks ago, with reply rates. The **interviews** sheet has one row per interview, with an interview_language column (the language it is held in) between interviewer and interviewer_email. The **offers** sheet (new 8 Oct 2026) has one row per opportunity offered to you: supervisor and email, project, country, dates, stipend, visa support, convention status, thread link, the stage you are in (offer_received, reply_drafted, questions_asked, accepted, convention_in_progress, convention_signed, declined, withdrawn) and the next step. `/pfe-inbox` updates it every run.

Opening Claude Code on the project

```
cd "C:\Users\<you>\Desktop\pfe workflow\campaign"
claude --chrome
```

Things you can simply say

You say	Claude does
`PFE-0123 submitted`	Moves a blocked row you applied to by hand back to applied, with a follow-up date.
`hold PFE-0500 until the LIST interview` / `release PFE-0393`	Adds or removes a line in holds.csv.
`PFE-0123 salary: say negotiable, submit it`	Answers a NEEDS_ANSWER question and submits.

`add to FORM_ANSWERS: postal code 1003`	Saves a standing answer for all future forms.
`/pfe-hunter Italy`	Hunts that country now instead of the next one in the rotation.

7. Every command

Command	Use	Email
`/pfe-hunter [Country] [TRACK or ALL]`	No argument: continues the current country of country_priority.csv until it is exhausted, then the next, track ALL. Covers every available opportunity in the country, no row or search limit, companies and startups first (incubators, accelerators, startup lists and registries, careers pages), then labs and universities; the report counts company and academic rows. With a country: hunts that one without changing the order.	none (tools disabled)
`/pfe-checker [n or ALL]`	Asks how many rows (any number or ALL, no maximum), then verifies them: bounced addresses first, then probable contacts, then discovered rows.	none
`/pfe-applier [n email] [n portal]`	Asks first, before reading anything, whenever the method or number is missing, even when a PROGRESS file exists: which method (email, portal or both; eligible rows per method and emails left under today's cap are shown in the later questions), then, for Portal, stops and lists every portal (the site of the application link) with its eligible rows and asks which portal(s) to apply through, one, several or all (17 Oct 2026), then how many applications of each (any number or ALL, no maximum). Only emails sent and forms submitted count; it keeps going to the next row of that method until the number is reached. Portal: blocked "no browser" rows, then web form rows; email: queued rows, then enriched A/B, then C with score ≥ 50. Rows of a method not chosen are left alone; an unfinished run resumes next time with the same choice. Emails go in batches of up to 10: the PDFs are copied once into `02_tracker\_upload\`, every draft is composed first (recorded as queued, draft composed), then one read of the Gmail Drafts list confirms the attachments and each draft that passes the checks is sent.	sends, after checks
`/pfe-inbox [FULL SWEEP]`	File and answer mail from the last 36 hours (FULL SWEEP: since 1 Sep), refresh the waiting list, send due follow-ups from it (max 15), add a row for each person a reply refers you to, expire dead rows.	simple replies and follow-ups
`/pfe-cleanup`	Monthly: dead rows to seen.csv.	none
`/pfe-send`	Interactive: decisions, held-back drafts and the drafts of `/pfe-applier ... draft`. Waits for your answer (`send H 1-20`, `send H all`).	only what you approve
`/pfe-send-held`	Sends yesterday's cap-held application drafts.	sends, after checks

`/pfe-interview-prep <email>`	Full interview preparation (.md + .pdf): full written answers, and a techniques section (every technique on your CV and used by the host, each with a sketch of its typical graph and labelled axes). Each interview gets its own folder, `C:\Users\<you>\Desktop\pfe workflow\interviews\<YYYY-MM-DD>_<Organisation>_<Interviewer>\` (date unknown: today and `_date-TBC`), with three files: the prep PDF, a short self-presentation deck (`<date>_<Organisation>_presentation.pptx`, modelled on your edited {{DECK_MODEL_SHORT}} deck: its slides only, and speaker notes that say the slide simply, then the result, with over-explanation between `//` and `//` for questions; since 17 Oct 2026) and `NOTEBOOKLM_PROMPT.txt`, a NotebookLM coaching prompt in the interview language that ends with a mock interview, one question at a time (since 13 Oct 2026). The prompt is always a plain-text note (.txt, no Markdown syntax), never .md (since 17 Oct 2026). The prep, the deck and the prompt never mention {{NEVER_MENTION}} or use it as an argument (version 12). Interactive.	none
`/forever <change>`	**Author only:** not installed in a campaign set up from sniper (locked install). Changes a rule permanently, in plain words: edits CLAUDE.md and the commands in campaign, then sniper (kit templates through make_kit.py and its privacy scan, setup prompt, runbook, guides, VERSION, CHANGELOG) and the copies in guides, backs up the old versions in archive\rule snapshots and checks nothing old is left.	none
`/magic [command]`	**Author only:** not installed in a campaign set up from sniper (locked install). Guided review of one command, task by task: shows the task map with line numbers, asks for each task keep, change, remove or explain more (type STOP to end early), checks the changes against CLAUDE.md, then applies the approved ones through `/forever`. Writes no tracker data, never touches Gmail, Chrome or portals.	none
`/ask-me <step>`	Help on one step you are stuck on: paste the step after the command. Claude finds it in the guides, checks the computer, does the step if it only needs commands or files (install check, winget, opening a file), otherwise explains it again as tiny numbered steps in simple words, with the check that shows it worked. Works in any folder (install.ps1 copies it to your own .claude\commands). Never signs in, never types a password, writes no tracker data. Every step of Friendly Setup and Usage ends with a line saying how to use it.	none

8. The safety rules: sending, forms, holds

When an email goes out by itself

Applications, follow-ups and simple inbox replies are sent with `send_message` on a finished Gmail draft (never by clicking Send in Chrome), and only if **all five** checks pass. Otherwise the draft waits and the report lists it. With the word `draft` (`/pfe-applier 20 email draft`) the Applier only makes the drafts and sends nothing: `/pfe-send` sends the ones you name, after the same checks.

1 Quality swap test, no banned words, no em dash → **2 New address** verified, never emailed before → **3 Not held** holds.csv, interviews, decisions waiting → **4 Attached** CV + letter PDFs on the draft, read on the Gmail Drafts list → **5 Under cap** < 50 cold emails today

NEVER AUTOMATIC

Replies about interviews, offers, salary, contracts, visa or legal questions are always A/B/C drafts for `/pfe-send` . And remember: **an email that went out cannot be unsent.**

UNPAID OR LOW-PAID OFFERS (10 OCT 2026)

You have no personal or {{SCHOOL}} funds. A host that would take you unpaid, or for a very low allowance, is negotiated, never declined, and no draft says you fund yourself. The drafts first look up what that position pays in that country (or Europe) and the anchors: the visa proof of means (Spain about 600 EUR a month), the legal intern minimum (France about 630 EUR a month in 2026), grants such as Erasmus+ KA171. Then they ask for one concrete monthly figure, up to the average plus 200 to 300 EUR when the host has the means, tied to an outside requirement ("The Spanish consulate asks for proof of about 600 euros per month..."), with alternatives: partial allowance, housing, a grant, a project contract. The report lists the figures and their sources.

Portal forms

Filled from `00_profile\FORM_ANSWERS.md` and submitted without asking. Saved choices (the setup asks them once): email always **{{GMAIL_ACCOUNT}}**; grades only as **{{GRADES_EN}}**; gender, date of birth, postal address, salary wording and diversity default as you answered (or NEEDS_ANSWER); {{NATIONALITY}}, needs a visa, handles it with the host's letter; {{SCHOOL}} signs the internship agreement; available {{PFE_WINDOW}}, 26 weeks.

The form asks for	What happens
Anything answerable truthfully from FORM_ANSWERS.md	Filled and submitted.
Passport/ID number or upload, GPA, a numeric salary, postal code, a legal visa declaration, or anything else not in the file	Not submitted. Row goes to blocked.csv as **NEEDS_ANSWER** with the exact question.
CAPTCHA, account creation, a login you are not signed in to, LinkedIn Easy Apply, Indeed Apply	Never attempted. Row goes to blocked.csv as **BLOCKED**.

Holds: stopping one company

`02_tracker\holds.csv` lists rows nothing automatic may send to or submit. Only you remove lines. Two holds are automatic and need no line: every other row of a company where you have an interview or offer, and any address already emailed once. Never apply to {{EXCLUDED_ORGS}}.

Rules that never change

- Never invent a contact, address, deadline or result; never build an email address from a pattern.

- Your own work stays separate from team work in every document.
- Grades only as {{GRADES_EN}}, nothing else: no other mark, average or exact rank.
- Maximum 50 cold emails a day; portals carry the volume.

9. What can go wrong, and how to fix it

You notice	Why	Fix
A step of the setup or a guide is unclear or does not work	Each step assumes the one before it worked.	Copy the step, type `/ask-me` in Claude Code, a space, paste it. Claude does it if it can, or explains it again in simpler words. Before Claude Code works, paste it at claude.ai.
`preflight.py` lists problems	A missing tool, Chrome profile or extension.	Fix what it names (section 3, "What no file can carry").
Applier report: "SESSION EXPIRED: GMAIL"	Google wants you to sign in again in Chrome.	Open Gmail in the {{SCHOOL}} Chrome profile and sign in. Claude never types passwords.
Hunter keeps hunting the same country	Normal: it stays until the country is exhausted.	To move on now, tell Claude "mark <country> exhausted in country_priority.csv". To reorder, "put <country> before <country>".
A run stopped part way	Plan usage limit reached, or the session filled up. Runs have no limit of their own, so big runs can reach it.	Run the same command again after the usage window resets. Rows are saved one by one; the Applier resumes from its PROGRESS file.
Many rows BLOCKED "LOGIN_WALL" / "SESSION EXPIRED"	A site logged you out, or Claude uses another Chrome profile.	Log in again in Chrome. `/chrome` → pick your campaign profile. Ask Claude to retry those rows.
Chrome actions fail, "extension not connected"	The extension is paused, or Claude Code is attached to another Chrome profile.	Check the extension in "{{CHROME_PROFILE}}": On all sites, Act without asking. In Claude Code: `/chrome` → Enabled by default, and pick the browser of {{CHROME_PROFILE}}.
Drafts land in the wrong Gmail account	Wrong `/mail/u/N/`.	Find which u/N is the {{SCHOOL}} account; tell Claude "Gmail compose is u/N".
"Leave site?" in Gmail, run stuck	Left a compose window while uploading.	Click it yourself; delete duplicate half-built drafts.
"Permission denied" in a log	Tool missing from the allow list.	Add it to `"allow"` in settings.json (one allow block only, commas between lines).
Someone refers you to a colleague	The reply gives a name and an address.	Nothing: Inbox adds a row, the Applier emails that person first and cites the referrer (not for auto-replies, or if they refused). A held company waits. A name with no address goes on your to-do list.
An email bounced	Dead address.	Nothing: Inbox notes it, Checker finds a new address.
A wrong or duplicate email went out	Rows under different domains, or a missing hold.	Add a hold now. Tell Claude; it notes it and can draft a short correction for `/pfe-send`.
An interview invitation arrived	Good news.	Answer in `/pfe-send`, then `/pfe-interview-prep <email>`. Same-company rows are held automatically.
Letters sound generic	Quality drift.	Read a few letters in `04_outbound`; tell Claude exactly what is wrong.
build.py: BANNED, or pages=2	Banned word, em dash, too long.	Applier rewrites and rebuilds by itself; persistent failures appear in the report.

PermissionError on tracker.csv	Excel or OneDrive locking it.	Close Excel. Never open tracker.csv during a run; read tracker.xlsx.
Rows look wrong or vanished	A bad write.	Every write left a copy in `02_tracker\backups\`. Say "restore tracker from the backup before <time>". Never edit the CSV by hand.
`/pfe-` commands missing	Claude Code opened in another folder.	`cd` into `C:\Users\<you>\Desktop\pfe workflow\campaign`, then `claude --chrome`.
`python` opens the Store	Python not on PATH.	Use the full path (section 10).
`!` command: "file does not exist"	Backslashes.	Use forward slashes.
Indeed tools unavailable	Indeed connector failed.	Hunter falls back to indeed.* in Chrome. Reconnect on claude.ai → Connectors.
Word dialog or frozen build	Word is stuck.	Close the dialog; end WINWORD.EXE in Task Manager.
A disabled plugin is needed	Disabled on purpose.	`claude plugin enable <name>@synced`

10. Reference

Folder map (`C:\Users\<you>\Desktop\pfe workflow\campaign\`)

Path	What it is
`CLAUDE.md`	Rules for every session (one merged rule set, version 24; read-only in a locked install). AGENTS.md is the Codex copy.
`.claude\commands\`	The 9 slash commands (eight /pfe-, /ask-me), read-only. The author's own campaign also has /forever and /magic.
`.claude\settings.json`	Permissions.
`00_profile\`	PROFILE, DEFENSIBLE_CLAIMS, TRACKS, KEYWORDS, FORM_ANSWERS, personalisation_map.json, interview_deck\ (build_deck.js, profile_deck.json, specs).
`01_sources\`	Source registry, {{SCHOOL}} registers, host patterns.
`02_tracker\`	tracker.csv (truth), next_actions.csv (what you must do yourself: type, due date, priority, open/done/dropped; first sheet of tracker.xlsx, since 22 Oct 2026), waiting_list.csv (applications with no reply yet: waiting, follow_up or followed_up; since 31 Oct 2026), interviews.csv, offers.csv, responses.csv, seen.csv, blocked.csv, holds.csv, tracker.xlsx (read-only view), tracker_tools.py, unblock_tools.py, backups\.
`03_documents\`	CV_MASTER_V2 (source of every tailored CV), LETTER_MASTER.
`04_outbound\<ID>\`	Each row's CV, letter, email.md, notes.md. Generator: `_gen\run34\build.py`.
`05_reports\`	Every run's report.
`scheduler\`	gate.py, preflight.py, snapshot.py (helpers you run by hand; nothing is scheduled).
`..\interviews\`, `..\my documents\`, `..\guides\`	One folder per interview; your source documents; the runbook and guides.
`..\sniper\`	The transfer package: setup prompt, kit, runbook, guides, build sources. Deleted for good at the end of the setup; only the author keeps it (and `/forever` keeps it in step).
`..\archive\`	Rule snapshots and parked files, never edited.

Control commands (type in Claude Code)

To	Type
Check a row count, then a run	`/pfe-checker ALL`, `/pfe-applier 15 email 10 portal`, `/pfe-hunter Portugal`
Which commands have work	`! "%LOCALAPPDATA%/Programs/Python/Python312/python.exe" scheduler/gate.py`
Tracker counts	`! cd 02_tracker && "%LOCALAPPDATA%/Programs/Python/Python312/python.exe" tracker_tools.py summary`
Stop everything	Do not start a command. Nothing runs by itself.

The settings.json that works

```
{
  "permissions": {
    "allow": [
      "Bash", "PowerShell", "Read", "Write", "Edit", "Glob", "Grep",
      "WebSearch", "WebFetch", "Skill", "Agent", "Task",
      "mcp__claude-in-chrome", "mcp__claude_ai_Gmail",
      "mcp__claude_ai_Google_Drive", "mcp__claude_ai_Google_Calendar",
      "mcp__claude_ai_Indeed",
      "mcp__claude_ai_Gmail__send_message", "mcp__claude_ai_Gmail__reply"
    ],
    "ask": [ "mcp__claude_ai_Gmail__forward" ],
    "deny": [
      "mcp__claude_ai_Gmail__trash_message", "mcp__claude_ai_Gmail__trash_thread",
      "mcp__claude_ai_Gmail__delete_label", "mcp__claude_ai_Gmail__mark_message_spam",
      "mcp__claude_ai_Gmail__mark_thread_spam", "mcp__claude_ai_Google_Drive__trash_file",
      "mcp__claude_ai_Google_Drive__share_file"
    ]
  },
  "env": { "PYTHONUTF8": "1" }
}
```

Key facts

Python	`C:\Users\<you>\AppData\Local\Programs\Python\Python312\python.exe` (set PYTHONUTF8=1)
Claude Code	`C:\Users\<you>\.local\bin\claude.exe`
Gmail account used to send	{{GMAIL_ACCOUNT}} (label `{{GMAIL_LABEL}}`)
Chrome profile	`--profile-directory="{{CHROME_PROFILE}}"` ({{SCHOOL}} account, has the extension)
Run size	No row, search, helper or spending limit; the tracker is updated after every row
Country order	`01_sources\country_priority.csv` (15 ranked countries, status per country)
Email cap	50 cold emails per day; portals unlimited

The highest-value move is still manual: write to the {{SCHOOL}} classmates listed in the registers who did their PFE at your target hosts. An introduction beats a cold email.